Cybersecurity by Design for Critical Infrastructures

Laurea Magistrale in Informatica

alma.pennacchi@uniroma2.it

Marzo 2026

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

Perché questo corso esiste?

Le infrastrutture moderne falliscono non perché attaccate, ma perché non comprese.

Problema reale:

- sistemi troppo complessi per essere compresi intuitivamente
- sicurezza applicata dopo il design
- monitoraggio senza modello

Obiettivo del corso

Imparare a progettare sistemi che siano sicuri perché comprensibili.

Roadmap

1 Introduzione

2 Laboratorio 1

Domanda guida

Quando un sistema è **sicuro**?

Domanda guida

Quando un sistema è **sicuro**?

Tesi del corso

Un sistema è sicuro quando il suo **comportamento atteso** è **formalizzato** e **verificabile**.

Obiettivi (oggi)

- Definire: **infrastruttura critica** e **sistema distribuito**
- Capire: **superficie osservabile** vs **superficie di attacco**
- Impostare il problema: **conoscenza** $\rightarrow$ **baseline** $\rightarrow$ **deviazioni** $\rightarrow$ **risposta**

Sistema distribuito vs Infrastruttura

Sistema distribuito

- nodi cooperanti
- comunicazione e coordinamento
- assenza di controllo unico

Infrastruttura

- sistema distribuito + **vincoli reali**
- compartimenti e domini
- interdipendenze IT/OT/cloud

Infrastruttura critica

Definizione operativa

Sistema distribuito la cui indisponibilità o compromissione produce **danni sistemici**.

- energia, sanità, acqua, trasporti, telecomunicazioni
- impatto: continuità servizi essenziali, safety, economia

Proprietà tipiche (che complicano la difesa)

- **distribuzione** geografica / edge
- **compartimentazione** (zone, conduits, DMZ)
- **legacy** e downtime costoso
- **supply chain** e manutenzione remota

Conseguenza

Non basta "mettere controlli": serve progettare la **misurabilità** del comportamento.

Superficie osservabile

Definition (Superficie osservabile)

Insieme delle informazioni estraibili da un sistema senza bisogno del payload applicativo.

- flussi: chi parla con chi, quanto, quando
- timing, periodicità, jitter
- errori, retry, handshake

Superficie di attacco

Definition (Superficie di attacco)

Insieme delle proprietà sfruttabili per impattare CIA e/o safety.

- servizi e protocolli esposti
- credenziali e trust boundary
- dipendenze operative (remote access, vendor, cloud)

Tesi fondamentale

Osservabilità $\Rightarrow$ potenziale attaccabilità

Non è equivalenza: è un'implicazione informativa.

- cifrare il payload non elimina metadati e pattern
- l'attaccante ottimizza: timing, selezione target, finestra operativa

Le 6 domande del corso

1. Cosa vogliamo **conoscere** di una infrastruttura?
2. Come rappresentiamo la **conoscenza** (ontologia/grafo)?
3. Qual è lo **standard** (baseline)?
4. Come identifichiamo **deviazioni**?
5. Come **classifichiamo** e attiviamo contromisure?
6. Come reagiamo a eventi **non classificati** ma "oggettivamente" malevoli?

Chiusura

Messaggio chiave

Se non modelli $\Rightarrow$ non misuri. Se non misuri $\Rightarrow$ non rilevi. Se non rilevi $\Rightarrow$ non difendi.

- giovedì: esempi e laboratorio (threat modeling su ospedale digitale)

Esempio (ICS) — Modello Purdue (schematico)

Livello 5/4 — Enterprise / IT
Directory, Email, ERP, SOC/SIEM, sistemi corporate

↓

Livello 3.5 — DMZ / Perimetro IT-OT
Proxy, jump host, repo patch, historian replica, broker dati

↓

Livello 3 — Operations / Site Control
SCADA servers, engineering workstation, asset inventory

↓

Livello 2 — Supervisory Control
HMI, supervisory control, batching, control room

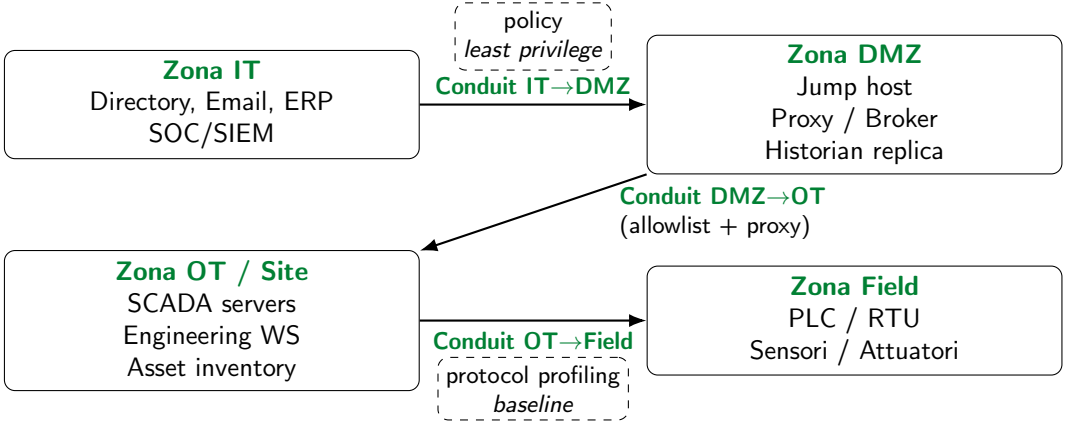
↓

Livello 1/0 — Field / Process
PLC/RTU, sensori, attuatori, fieldbus

Confine critico
zona/conduit
policy-driven

Osservabili
flussi, timing,
topologia implicita

Zone & Conduits (IEC 62443) — perché servono



Collegamento al corso

Le zone riducono la complessità: trasformano una rete in un insieme di **comportamenti attesi** verificabili. Le conduits diventano punti naturali per **osservazione**, **baseline** e **contromisure**.

Esempio: inferenza senza payload

- frequenza messaggi $\uparrow$
- latenza $\uparrow$
- retry $\uparrow$

Esempio: inferenza senza payload

- frequenza messaggi ↑
- latenza ↑
- retry ↑
- **fault** / degrado
- **stress operativo**
- **attacco** (DoS, scanning, lateral movement)

Formalizzazione minimale

Obiettivo

$$F(I,t) \to s, \quad s \in \{\text{normale},\ \text{anomalo},\ \text{attacco},\ \text{fault}\}$$

- I include: asset, relazioni, vincoli, processi
- F dipende da: osservabili, modello, baseline, policy

Roadmap

1 Introduzione

2 Laboratorio 1

Lab 1 — Threat Modeling

Obiettivo

Costruire un modello iniziale di sicurezza: **asset** → **osservabilità** → **inferenza** → **attacco**.

- gruppi da x
- consegna: PDF

Scenario: ospedale digitale

Componenti:

- EHR / cartelle cliniche (server + DB)
- Wi-Fi ospiti/pazienti
- dispositivi medici IoT/OT
- prenotazioni (cloud)
- VPN medici remoti

Vincoli reali

continuità servizio, safety, legacy, manutenzione remota

Step 1 — Asset inventory

Compilare:

Asset	Tipo	Criticità	Dipendenze

Step 2 — Superficie osservabile

Per ogni asset: indicare (senza payload)

- cosa osservo
- cosa inferisco

Euristica

chi comunica con **chi**, **quando**, **quanto**, **con che regolarità**.

Step 3 — Inferenza → Attacco

Compilare:

Asset	Osservabile	Inferenza	Attacco

Step 4 — Grafo dell'infrastruttura

Disegnare un grafo:

- nodi = asset
- archi = comunicazioni
- opzionale: protocollo e frequenza

Perché il grafo?

Baseline topologica + discovery + deviazioni + propagazione del rischio.

Debrief

Domande:

1. asset più **critico**?
2. asset più **osservabile**?
3. asset più **pericoloso** come pivot?
4. quale controllo by design mettereste per primo?

Chiusura

Il sistema più vulnerabile è quello **meno modellato**.